



**Institut supérieur de
mécanique de Paris**

Rapport PSYN

Analyse d'image appliquée à des matériaux composites
soumis à des essais tribologiques

FISA - ISAE-Supméca



Laboratoire Euler - ISAE-Supméca

Auteur : Oscra MA, Hugo WEIBEL

Encadrant : M. FORTES DA CRUZ, Mme. LO FEUDO

Date : Janvier 2026

Année universitaire 2025–2026

Sommaire

I.	Introduction	2
II.	Description de l'algorithme	3
III.	Le programme	17
IV.	Utilisation du Machine learning	25
V.	Conclusion	26
A	Algorigramme	29
B	Configuration pour le <i>sample 25</i>	30

I. Introduction

Ce projet porte sur l'étude d'échantillons d'un matériau composite appelé *HexTOOL*, breveté par la société *Hexcel Composites*, il est principalement utilisé dans la fabrication de moules destiné à la production de pièces aéronautiques. Ce matériau se comporte globalement de manière quasi-isotrope mais présente des disparités locales dans l'orientation de ces fibres, il est donc nécessaire d'identifier et de localiser les zones d'orientation préférentielle de fibres. Jusqu'à présent la détection/délimitation de ces zones se faisait manuellement. Cela est très chronophage et peut être moins précis qu'une méthode numérique. Ainsi le principal objectif de ce projet est d'automatiser la délimitation de ces zones d'orientation préférentielles, grâce à des méthodes d'analyse d'images.

Le but de ce rapport est de présenter la méthode principale que nous avons implémentée et les résultats que nous avons pu obtenir. Dans un premier temps nous présenterons notre approche principale, puis nous montrerons les différents essais et les résultats obtenus. Par la suite nous verrons les autres méthodes qui peuvent être envisagées. Enfin nous conclurons en abordant les critères de qualité requis pour le bon fonctionnement de notre algorithme et les points d'amélioration de notre méthode.

Ce projet utilise principalement la version Python de la librairie `OpenCV`. Cette dernière est totalement open source et est spécialisée dans le traitement d'images. Par conséquent l'entièreté de notre code source est disponible en open-source sur GitHub à l'adresse suivante : <https://github.com/Brawzzz/PSYN>

II. Description de l'algorithme

Cette section explique en détails l'algorithme que nous avons implémenté. Nous détaillerons les objectifs de cette dernière mais aussi les structures et les techniques mise en place pour son développement. L'ensemble des étapes nécessaire a notre algorithme sont décrites dans l'algorigramme en annexe

1. Objectifs et structures

Notre méthode repose entièrement sur la détection des fibres contenus dans les échantillons. Dans un premier temps, on tente de détecter le plus de fibres via une détection de contours, une méthode classique en analyse d'image. Puis on effectue une classification en fonction de l'orientation de chaque fibre trouvée. On obtient alors des groupes correspondant aux zones d'orientation préférentielle que l'on souhaite délimiter de manière géométrique.

Afin d'implémenter cet algorithme nous avons décidé d'appliquer la méthode « diviser pour mieux régner », en travaillant sur des sous-images de l'image d'origine. Cela permet principalement de faciliter les calculs mais aussi de permettre leur parallélisation. De plus nous avons utilisé une approche orientée objet. Cela permet de décrire de manière précise et intuitive les différents objets que l'on manipule, de plus, la structure du code source devient plus claire et plus maintenable. La structure que nous avons conçue est composée des éléments suivants :

- **Fiber** : est la classe élémentaire, qui décrit les fibres contenues au sein du matériau. Elle est définie principalement par un contour (série de points) et un angle (0–180°)
- **Region** : est la classe qui décrit les zones d'orientation préférentielle. Elle est définie par une liste de *Fiber*.
- **Sample** : Un *Sample* décrit les résultats obtenus par l'analyse tribologique et ceux par analyse d'image. Il est défini par une image et une liste de *Region*
- **Split** : est une classe qui correspond à un sous *Sample*. Cette classe est artificielle dans le sens où elle ne décrit pas réellement un objet physique, mais permet de faciliter à la fois le calcul et le rendu des *Regions*.

Une telle structure, utilisant une approche objet, permet d’abord une plus grande compréhension de notre problématique mais aussi de rendre le code source plus intuitif et surtout plus maintenable. La structure et les relations entre les différentes classes sont décrites par le diagramme suivant :

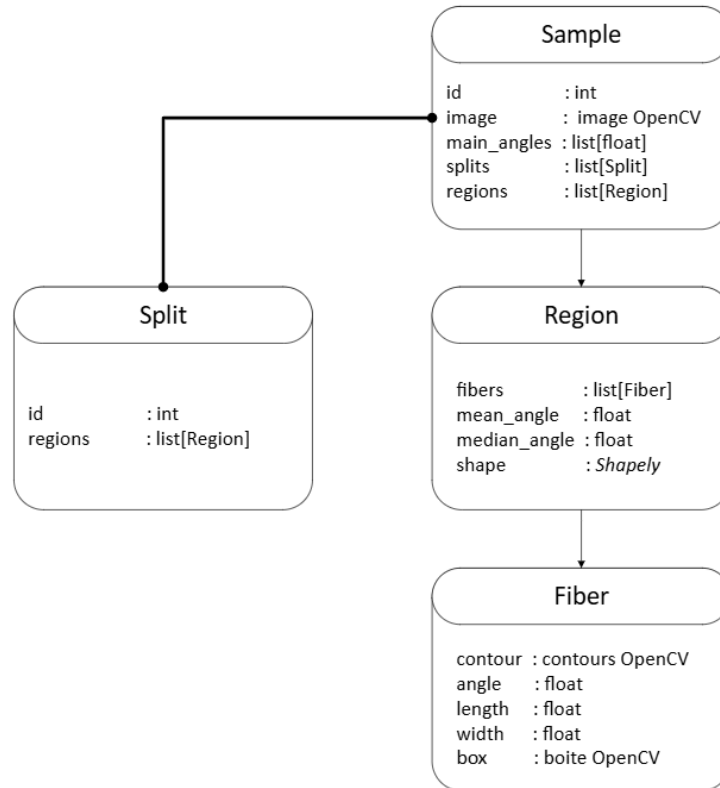


FIGURE 1 – Structure du classe

2. Détection et sélection

La détection des fibres est basée sur de la détection de contours qui est une méthode standard en analyse d’image. Cette dernière se déroule en trois étapes : floutage, seuillage, détection.

Le floutage est une opération de base en traitement d’image. Le but est

d'éliminer le bruit contenu dans une image. L'outil mathématique utilisé pour cela est le plus souvent une simple moyenne pondérée. On remplace la valeur de chaque pixel par la moyenne pondérée de celles de ses voisins. Ainsi la plupart des fonctions OpenCV qui réalisent ce genre d'opérations diffère seulement du poids utilisé dans le calcul de la moyenne.

$$I_f(p) = \frac{1}{W_p} \sum_{q \in V} w(p, q) \cdot I(q)$$

avec :

$I(p)$: l'intensité du pixel p

$I_f(p)$: l'intensité filtré du pixel p

$I(q)$: l'intensité du pixel q

$w(p, q)$: le poids associé au pixel p par rapport au pixel q

W_p : le facteur de normalisation (somme des poids)

Nous avons commencé par utiliser la fonction `cv.GaussianBlur()` pour le floutage. Ici le poids utilisé ne dépend que de la distance géométrique aux pixels voisins. On a alors :

$$w(p, q) = (||p - q||)$$

Nous avons ensuite décidé d'utiliser la fonction `cv.bilateralFilter()`. Cette fois le calcul du poid se fait en combinant la distance géométrique (flou Gaussien) et la différence d'intensité des pixels voisins. On obtient alors l'équation suivante :

$$w(p, q) = (||p - q||) \cdot (||I(p) - I(q)||)$$

Le filtrage bilatéral présente deux intérêts majeurs : il ne décale pas le bord réel des fibres, contrairement au flou Gaussien, et supprime les textures internes qui pourraient créer de faux contours que l'on pourrait considérer comme des fibres. C'est pour ces deux raisons que nous avons retenu la fonction `cv.bilateralFilter()` pour réaliser l'étape de floutage. La figure suivante montre le résultat de ce floutage sur un *split* :

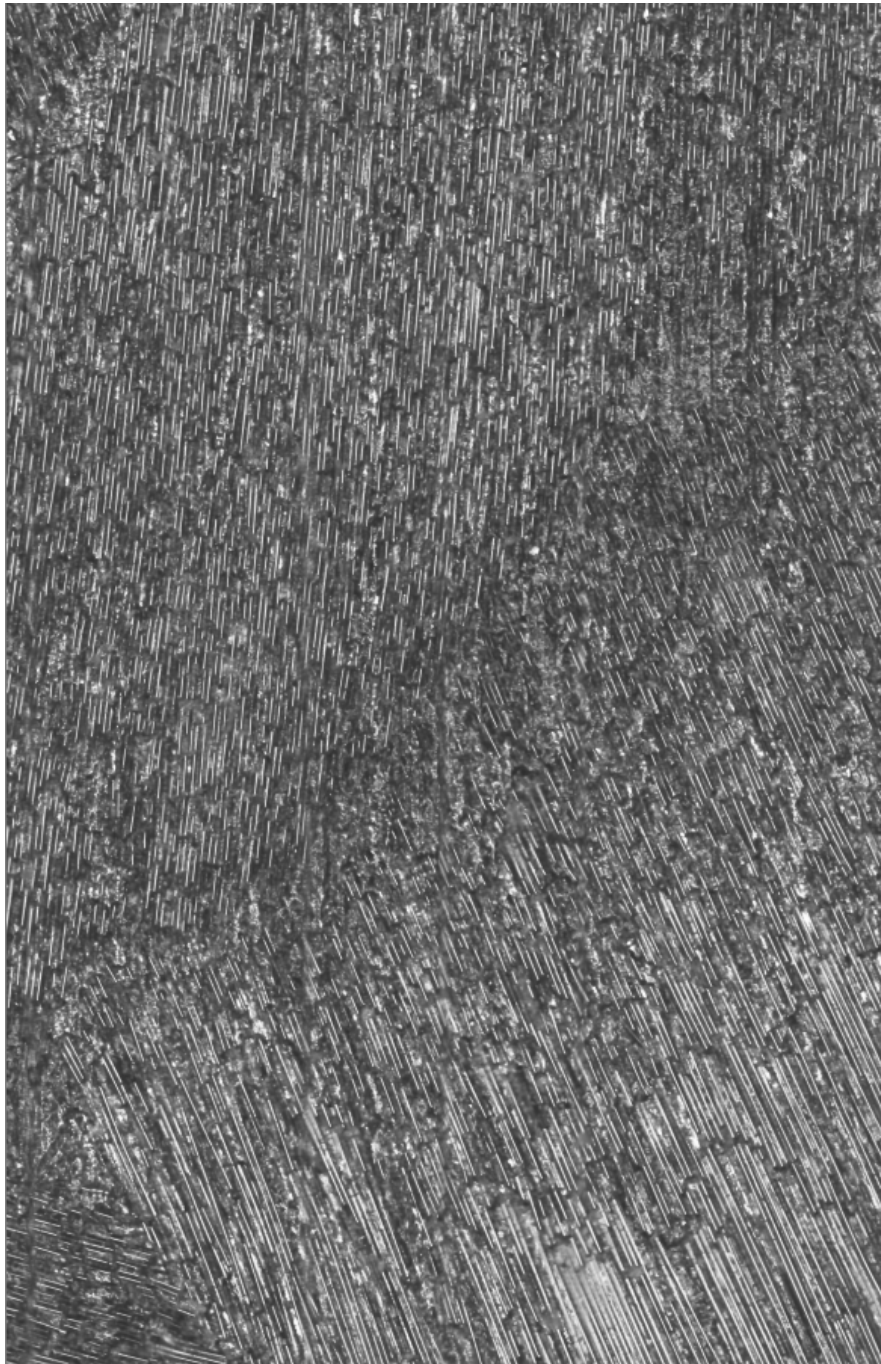


FIGURE 2 – Résultat du floutage bilatéral sur le *split* 14/64

Le seuillage, ou encore binarisation, est une opération locale, qui consiste à rendre une image initialement en niveaux de gris, binaire. Cela signifie que chaque pixel de l'image devient soit blanc (255) soit noir (0). Les fonctions qui réalisent ce genre d'opération appliquent une règle simple : « *si le pixel est plus lumineux qu'un certain seuil, il devient blanc. Sinon, il devient noir.* »

La fonction de seuillage que nous avons choisie d'utiliser est `cv.threshold()`, qui attend en paramètre le seuil et le type de binarisation. Ici le paramètre *type* est défini par :

$$type = cv.THRESH_BINARY + cv.THRESH_OTSU$$

L'option : `cv.THRESH_BINARY` permet un seuillage binaire classique et l'option : `cv.THRESH_OTSU` permet de calculer automatiquement le meilleur seuil possible en fonction de la forme de l'histogramme de l'image. L'algorithme suppose que l'image à binariser ne contient que deux classes de pixels, le premier plan et l'arrière-plan. Il calcule ensuite le seuil optimal qui sépare ces deux classes afin que leur *variance intra-classe* soit minimale :

$$\sigma_w(t)^2 = w_1(t) \cdot \sigma_1(t)^2 + w_2(t) \cdot \sigma_2(t)^2$$

En somme, le type de binarisation `cv.THRESH_OTSU` consiste à minimiser la variance de chaque groupe de pixels, celui correspondant à l'arrière plan ($\sigma_1(t)$) et au premier plan ($\sigma_2(t)$).

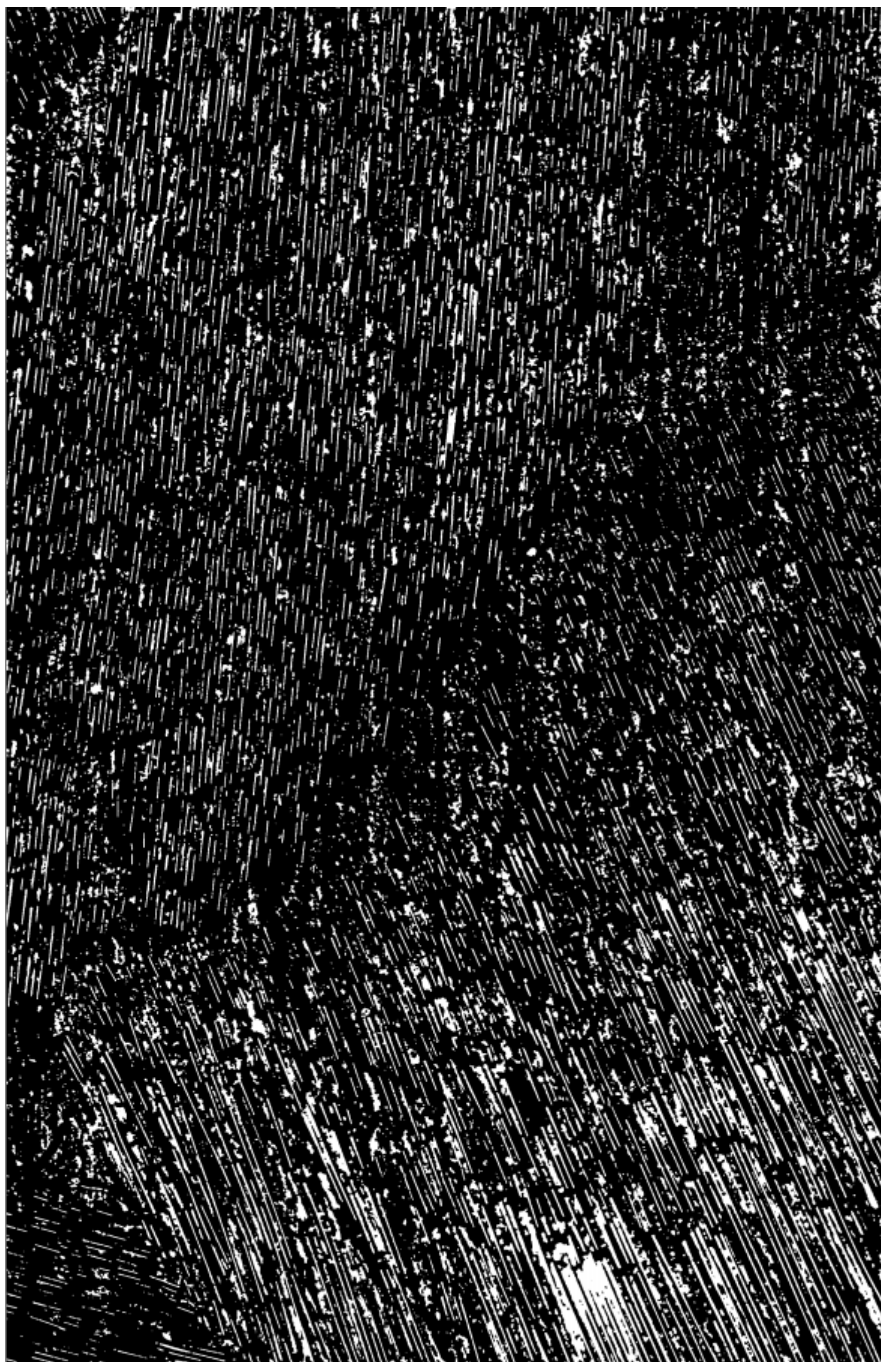


FIGURE 3 – Résultat du seuillage sur le *split* 14/64

La détection des bords est réalisée par la fonction `cv.findContours()`. Cette dernière permet la détection de contours sur une image binaire ainsi que le passage du niveau traitement d'image (gestion de pixels) au niveau du traitement vectoriel (gestion de points, vecteurs ...). Cette fonction nous retourne donc les contours, contenus dans une image binaire, dans un format vectoriel, c'est-à-dire un contour sous forme d'une liste de points. Une fois la détection réalisée, il est nécessaire de sélectionner les contours qui correspondent réellement à des fibres. Pour cela on ne conserve que les contours qui valident le test suivant :

- `perimeter > stp.FIBER_PERIMETER_MIN`
- `length >= stp.FIBER_LEN_MIN`
- `width <= stp.FIBER_WIDTH_MAX`
- `ratio > stp.FIBER_RATIO_MIN`

Les dimension *perimeter*, *length*, *width*, *ration* sont extraites de la boite englobante orientée du contours.

Lorsque que nous avons effectué cette vérification il reste encore des faux positifs, des contours qui semblent être des fibres valides géométriquement mais qui se trouvent dans des zones où il n'y en a pas et qui ne décrivent pas le contours d'une fibre réelle. Pour éliminer ces derniers contours, on applique l'algorithme *DBSCAN* (*Density Based Spatial Clustering of Application with Noise*), un algorithme de partitionnement qui se base sur la densité pour créer des clusters dans un nuage de points.

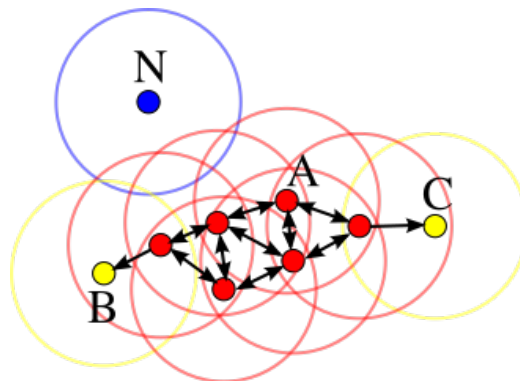


FIGURE 4 – Illustration de l'algorithme *DBSCAN*

Une fois que le nuage de points a été partitionné on ne conserve que les points situés dans des clusters suffisamment denses. Par exemple sur la figure ci-dessus on ne sélectionnerait que les points dans les clusters A , B , C . On applique donc cet algorithme à l'ensemble des points de tout les contours trouvés au sein d'un *split*. Dans l'exemple ci-dessous on ne conserve que les fibres dans les clusters rouge et jaune pour les fibres vertes, et on applique la même méthode pour les fibres bleues :

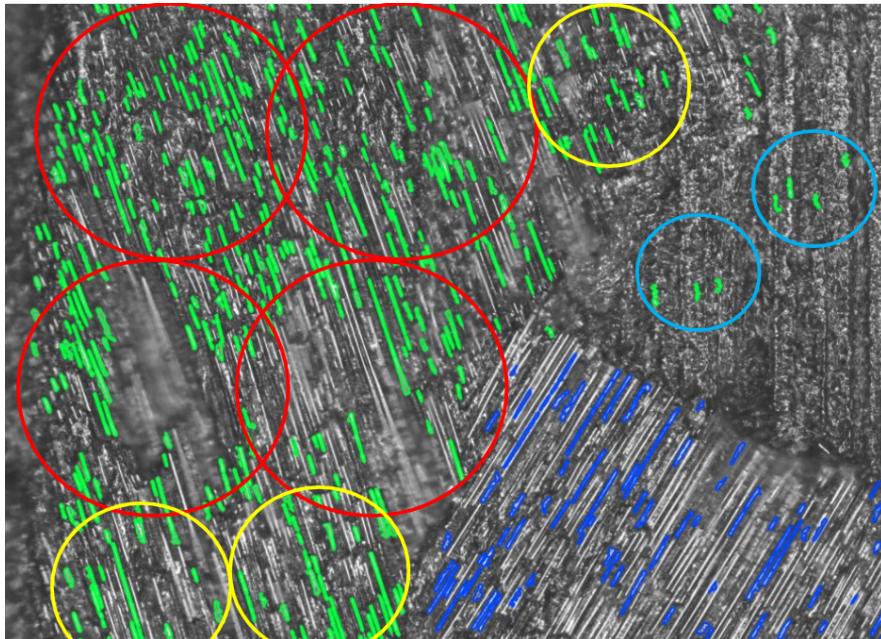
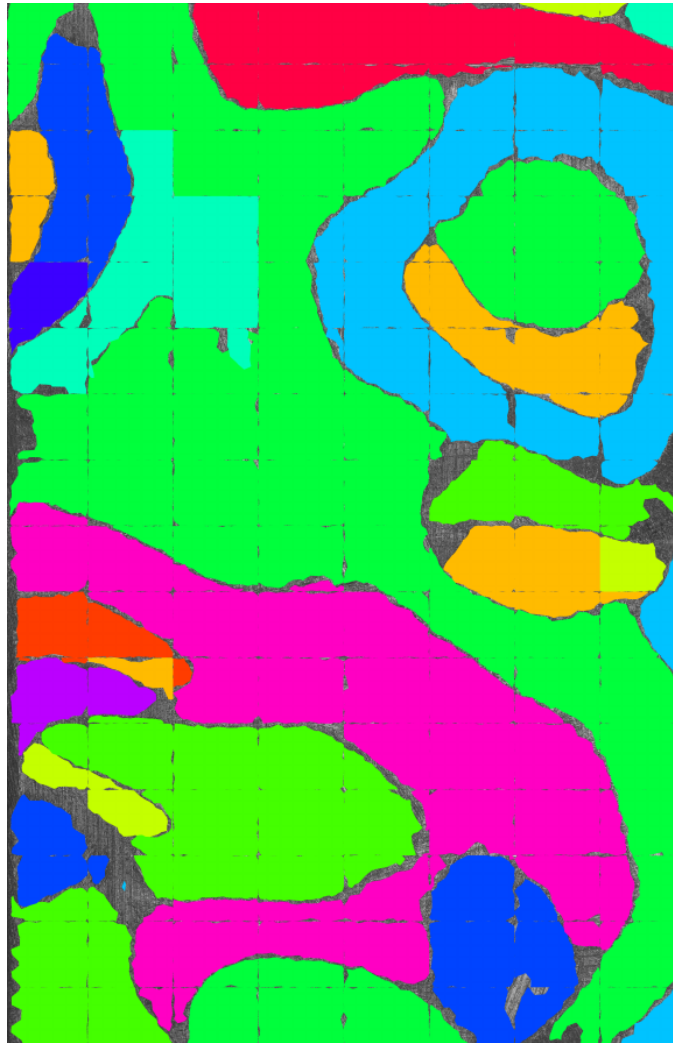


FIGURE 5 – Application de l'algorithme *DBSCAN* sur le *split* 1/128

3. Classification

Une fois que nous avons détecté et sélectionné le plus de fibres cohérentes possible, nous devons réaliser une classification de ces dernières. On souhaite regrouper les fibres qui ont la même orientation entre elles pour pouvoir définir des régions d'orientations préférentielles. Dans un premier temps nous avons fait cette opération de manière locale, au sein de chaque *split*. Enfin on reconstruit l'image complète en concaténant tous les *splits*.

FIGURE 6 – Rendu des régions par *split*

Cette méthode présente plusieurs inconvénients : d’abord cela crée de légers espaces vides correspondant à l’espace entre chaque split. Ensuite on ne calcule pas les zones d’orientation globales mais seulement localement. Pour pouvoir corriger ces défauts nous avons décidé de faire le calcul de manière globale, c’est-à-dire que l’on continue à détecter et sélectionner les fibres localement mais on effectue un mapping pour obtenir les coordonnées de leur contours dans le repère de l’image complète.

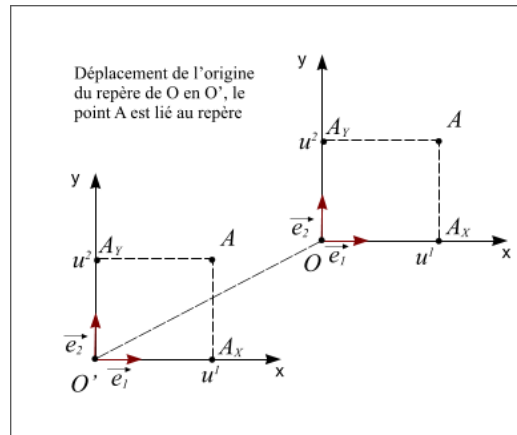


FIGURE 7 – Mapping par translation

Une fois cette opération réalisée nous avons une liste de toutes les fibres dans le repère de l'image globale, que l'on souhaite classifier selon leurs orientations. Pour cela on commence par calculer l'histogramme des angles, c'est-à-dire déterminer le nombre de fibres par orientation.

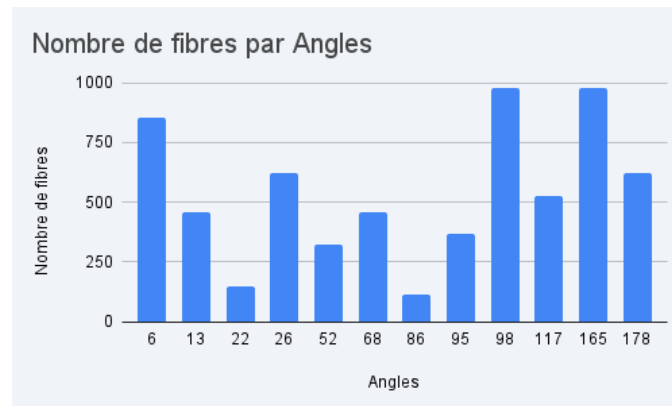


FIGURE 8 – Diagramme du nombre de fibre par angles

A partir de cet histogramme nous allons pouvoir extraire les directions principales, c'est-à-dire les angles qui regroupent le plus de fibres. Une fois que nous avons pu obtenir cette liste d'angles principaux, nous allons affecter chaque fibre à la direction principale la plus proche de sa direction propre.

On obtient alors plusieurs groupes de fibres qui définissent les *Regions* globales.

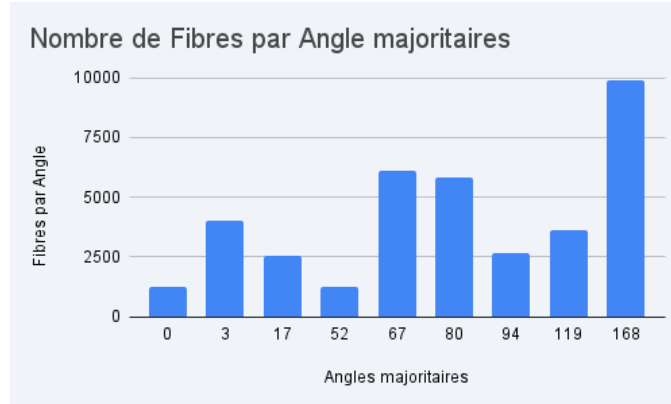


FIGURE 9 – Diagramme du nombre de fibre par direction principales

4. Délimitation

La délimitation des zones d'orientations préférentielles est la dernière étape de notre algorithme. Une fois que nous avons construit les différentes *Regions* avec l'ensemble de nos fibres, nous voulons les délimiter, déterminer leur contour géométrique. La première méthode à laquelle on pense pour répondre à ce genre de problématique est de calculer l'enveloppe convexe. Cette approche présente deux problèmes majeurs : d'abord l'enveloppe convexe est un polygone, on ne sera donc incapables de suivre précisément le pourtour de ces zones. Ensuite les régions formées par les fibres au sein de ce matériau ne sont pas nécessairement convexes. Ainsi une telle description ne correspond pas du tout à notre problématique.

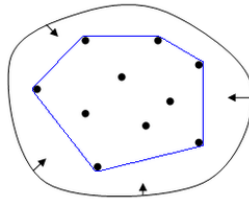


FIGURE 10 – Enveloppe convexe d'un nuage de points

Nous avons alors testé deux autres méthodes permettant de calculer des contours non convexes et même « *troués* » à partir de nuage de points. Le nuage de points que l'on considère est l'ensemble des points des contours des fibres appartenant à une *Region*.

Le premier algorithme que nous avons utilisé est appelé *Alpha shape*. Il repose sur la théorie des graphes et la géométrie computationnelle, c'est en réalité un sous-graphe de la *Triangulation de Delaunay*. Il se déroule en trois étapes :

1. *Triangulation de Delaunay* : L'algorithme commence par réaliser la triangulation de Delaunay sur le nuage de points. L'objectif est de former des triangles, de manière à ce qu'aucun point ne se trouve à l'intérieur du cercle circonscrit de n'importe quel triangle. Cela crée un maillage complet qui couvre tout l'espace.

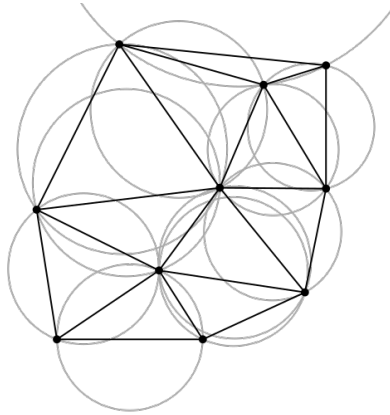


FIGURE 11 – Triangulation de Delaunay

2. *Filtrage par rayon* : Pour chaque triangle du maillage, on calcule son rayon circonscrit r , c'est-à-dire le rayon de son cercle circonscrit. Le paramètre α définit le seuil critique, on supprime tous les triangles dont le rayon est trop grand. Critère de suppression : Si $r > \frac{1}{\alpha}$, le triangle est supprimé. Ce critère s'explique par la réflexion suivante : si un triangle est « grand » cela signifie que ses 3 points sont très éloignés les uns des autres, et que cela représente du « vide » ou une zone de faible densité.

3. *Tracé du contour* : Une fois que les triangles « trop grands » sont supprimés, il ne reste que les « petits » triangles denses. L'union de ces triangles forme l'aire de la région, et les arêtes de ces triangles qui n'ont plus de voisin deviennent le contour extérieur, ce que l'on cherche. En résumé c'est le paramètre α qui décide de la précision de l'enveloppe. Si $\alpha = 0$, on obtient l'enveloppe convexe du nuage de points. Si $\alpha > 0$, on obtient une enveloppe plus « serrée » autour du nuage de points.

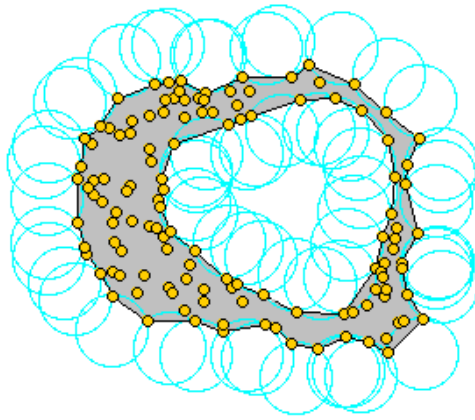


FIGURE 12 – Illustration de l'algorithme *Alpha shape*

La seconde méthode que nous avons utilisée s'appelle une fermeture morphologique vectorielle, que l'on implémente avec une librairie spécialisée dans ce genre d'opération appelée *Shapely*. Cet algorithme est composé de trois étapes :

1. *Le Buffering Positif* (Dilatation) : pour chaque point p du contour de chaque fibre, on définit un disque de rayon R . L'ensemble de tous ces disques forme la nouvelle géométrie dilatée de la fibre. Cette opération a pour conséquence de modifier la topologie de l'ensemble des contours. Certains polygones disjoints deviennent sécants. Les détails fins, inférieurs à R sont remplis.
2. *L'Union Booléenne* : L'algorithme calcule les intersections exactes des arcs de cercle générés par le buffer et supprime toutes les frontières internes. Il fusionne les géométries pour n'en faire qu'une seule valide.

3. *Le Buffering Négatif* (Erosion) : on applique un offset négatif à la forme fusionnée obtenue à l'étape précédente. Ici le point clé est que l'érosion n'est pas l'inverse parfait de la dilatation.

$$Dilatation(Erosion(Shape)) \neq Shape \quad (1)$$

$$Erosion(Dilatation(Shape)) \approx Shape \quad (2)$$

Tous les trous ou espaces vides qui ont été entièrement couverts lors de l'étape de dilatation ne réapparaîtront pas lors de l'étape d'érosion. Le bord extérieur, lui, recule de la distance R , fusionnant ainsi tangentiellement avec les fibres les plus extérieures.

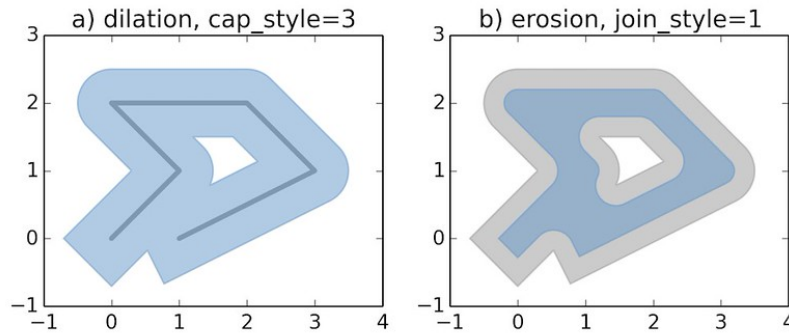


FIGURE 13 – Illustration de la fermeture morphologique

III. Le programme

1. Utilisation

Notre algorithme nécessite plusieurs librairies Python pour pouvoir s'exécuter correctement. La procédure permettant l'utilisation de ce programme est détaillé dans le fichier *ReadMe.md* sur GitHub. De plus le script s'appuie sur plusieurs paramètres, dédiés aussi bien au calcul qu'au rendu. Ces paramètres sont les suivants :

- *sample_index* : désigne l'indice de l'échantillon (*Sample*) étudié
- *nb_split* : nombre d'imagette (*Splits*)
- *delta_angle* : correspond a l'intervalle dans lequel deux angles sont considérés comme égaux

$$\alpha_1 = \alpha_2 \Leftrightarrow \alpha_2 - \delta \leq \alpha_1 \leq \alpha_2 + \delta$$

- *blur_method* : méthode de floutage utilisée (BILATERAL, GAUSSIAN)
- *thresh_method* : méthode de seuillage utilisée (CLASSIC, ADAPTATIVE)

Les paramètres suivant gèrent les caractéristiques géométriques d'une fibre :

- *fiber_perimeter_min* : périmètre minimum
- *fiber_len_min* : longueur minimum
- *fiber_width_max* : largeur maximum
- *fiber_ratio_min* : allongement minimum

On définit de la même manière des contraintes sur les dimensions d'une région :

- *region_min_fiber* : nombre de *Fiber* minimum pour considérer une *Region* valide
- *region_min_area* : aire minimum (en pixel) pour considérer une *Region* valide
- *hole_min_area* : aire minimum (en pixel) pour considérer un « trous » valide

Les paramètres ci-dessous définissent les réglages pour la délimitation des *Regions* :

- *shape_method* : méthode de délimitation utilisée (MORPH, A_SHAPE)
- *r_dilate* : taux de *dilatation* pour la fermeture morphologique
- *r_erosde* : taux de *erosion* pour la fermeture morphologique
- *alpha* : définit le paramètre α pour l'algorithme *Alpha Shape*
- *point_step* : pas de points utilisé pour l'algorithme *Alpha Shape*, la méthode étant très coûteuse on peut choisir de ne considérer qu'une certaine partie des fibres pour accélérer le calcul.

Enfin ces derniers paramètres contrôlent le rendu :

- *render* : type de rendu souhaité
- *fill_shape* : choisir de remplir les *Regions* (true/false)
- *shape_thickness* : épaisseur du tracé des délimitations
- *fiber_thickness* : épaisseur du tracé du contours des *Fiber*
- *background* : définit le fond pour le rendu (black, white, real)

L'utilisateur définit ces paramètres dans un fichier de configuration au format JSON. Ce fichier sert d'entrée principale au programme pour générer les résultats correspondant à la configuration choisie. Le script génère alors plusieurs fichiers en sortie :

- Fichiers .roi : une liste de fichiers correspondant aux contours de chaque région identifiée.
- Fichier de log : un fichier synthétisant l'ensemble des paramètres utilisés, permettant ainsi de conserver et de reproduire une configuration spécifique.
- Rendus graphiques : l'exportation au format .png de toutes les *Regions*, individuellement et collectivement.

2. Essais et résultats

La problématique majeure de notre méthode est la configuration des paramètres. Cette étape est relativement longue et dépend complètement du

sample étudié. Nous nous sommes donc principalement concentrés sur le *sample 25* pour les différents tests et rendus. La configuration des paramètres retenues pour ce dernier est présentée en annexe.

Cette section a donc pour objectif de présenter les rendus obtenus que nous considérons comme les plus satisfaisants, avec la configuration que nous avons retenue pour le *sample 25*. Ainsi la série de figures ci-dessous montre successivement différents type de rendus obtenus avec la fermeture morphologique puis Alpha Shape :

1. La délimitation des régions
2. La délimitation des régions avec fibres
3. Le rendus d'une *Region* seule

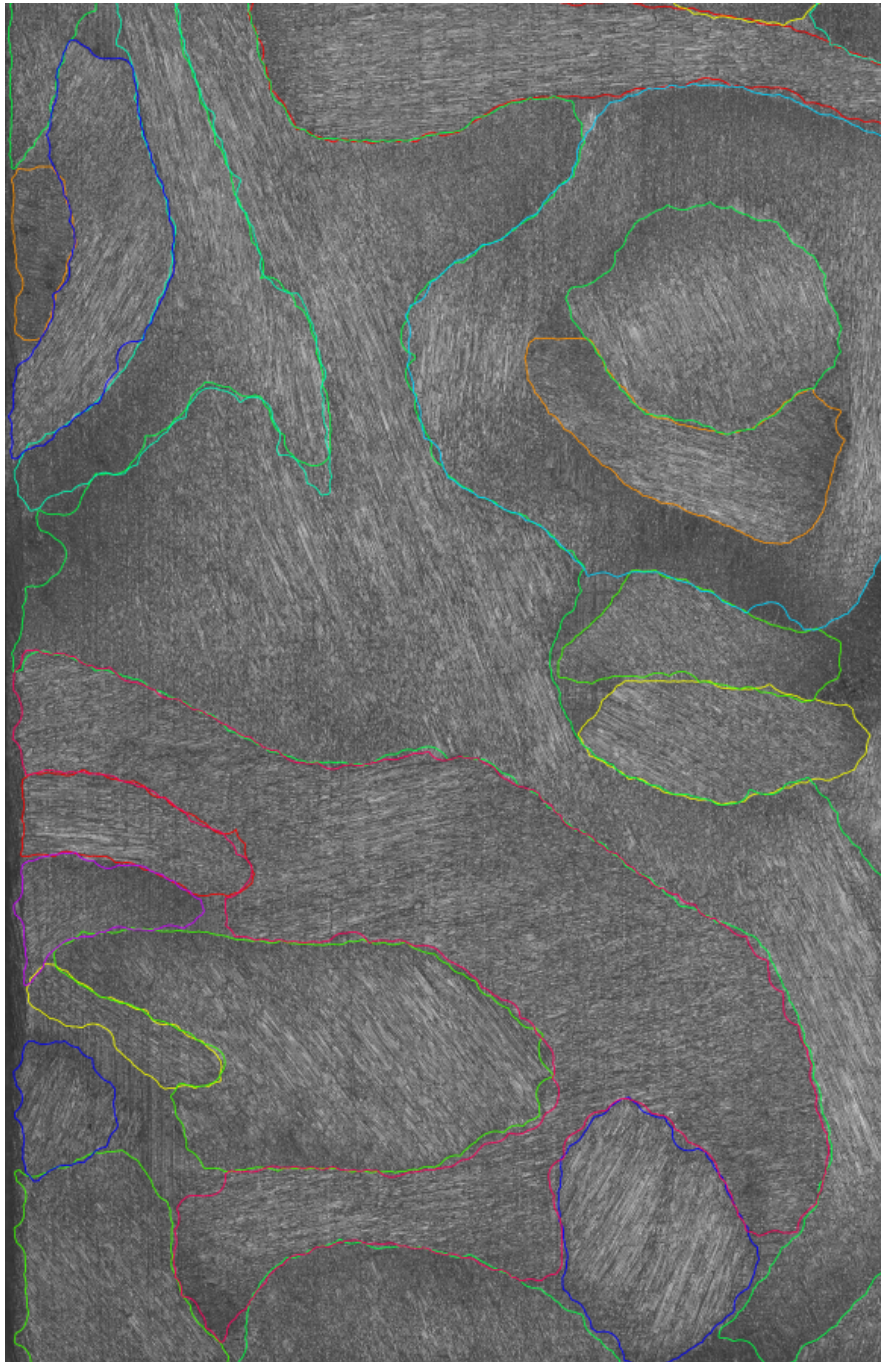


FIGURE 14 – Rendu des délimitations par fermeture morphologique



FIGURE 15 – Rendu des délimitations avec fibres par fermeture morphologique

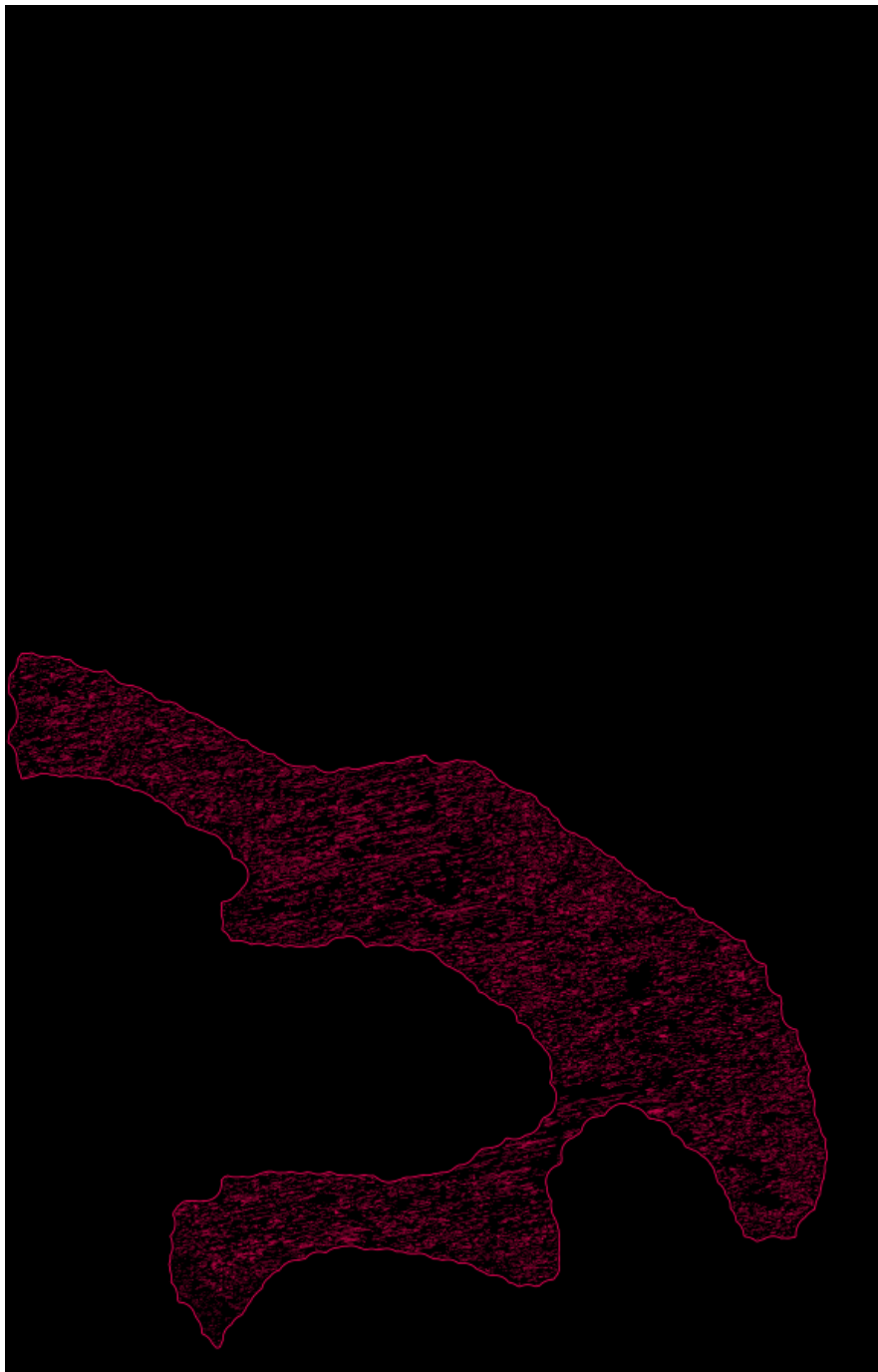


FIGURE 16 – Rendu d'une *Region* seule par fermeture morphologique



FIGURE 17 – Rendu des délimitations et des fibres avec Alpha Shape

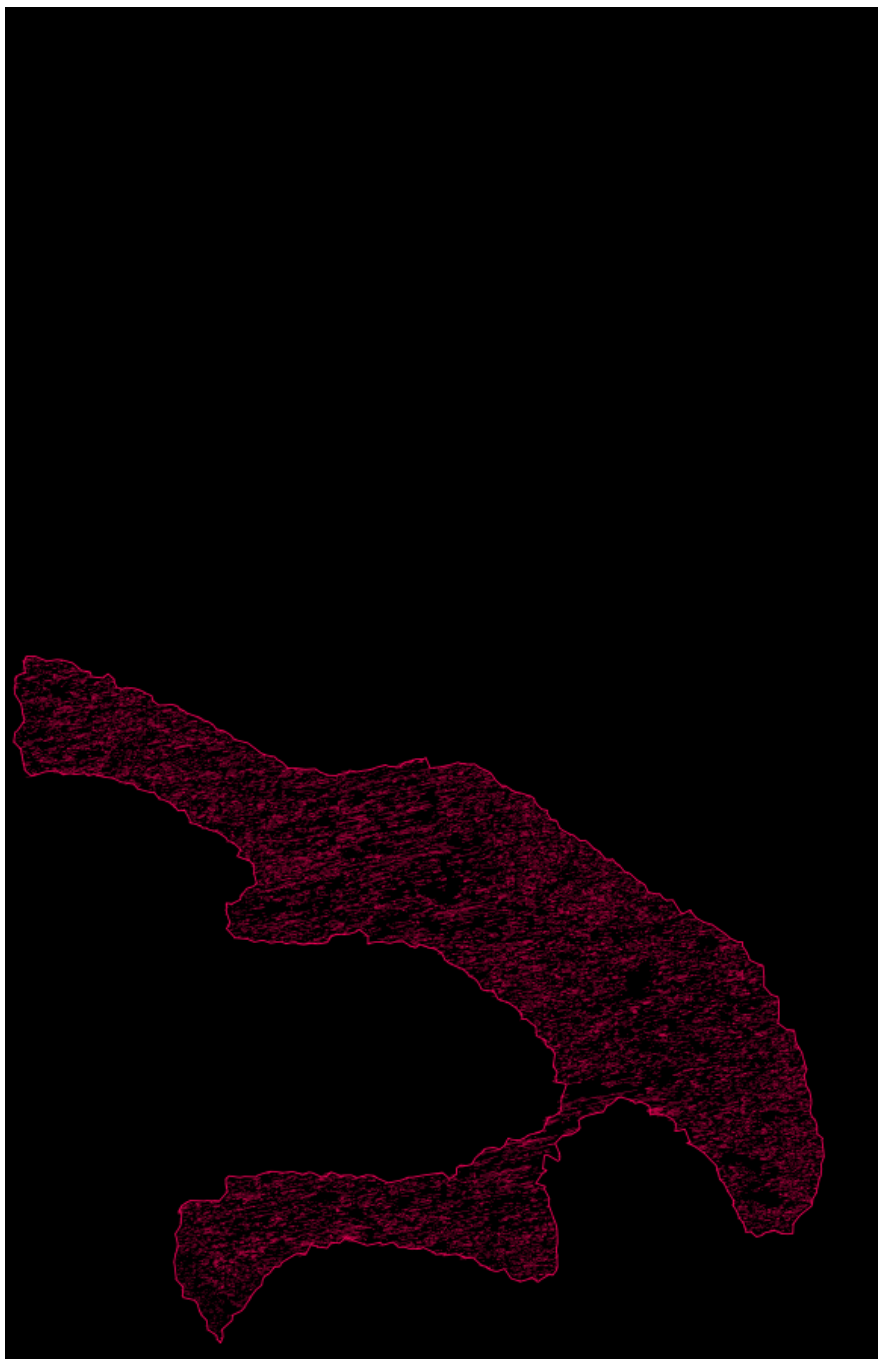


FIGURE 18 – Rendu d'une *Region* seule avec Alpha Shape

IV. Utilisation du Machine learning

Le contexte de ce projet est particulièrement adapté à l'utilisation du Machine Learning ; en effet, la notion de réseaux neuronaux convolutionnels est très répandue dans les domaines de la Computer Vision et du traitement d'images. C'est pour cette raison que nous avons envisagé d'utiliser ces technologies pour rendre notre algorithme encore plus efficace. D'abord, nous voulions tenter d'implémenter les modèles suivants pour améliorer la détection des fibres :

Critère	U-Net	YOLOv8-seg	Mask R-CNN
Type de tâche	Segmentation Sémantique	Segmentation d'Instance	Segmentation d'Instance
Fibres collées	Les fusionne (post-traitement)	Les sépare très bien	Les sépare parfaitement
Quantité de données	Faible (20-30 images)	Moyenne	Élevée
Vitesse d'exécution	Rapide	Ultra-rapide	Très lent
Implémentation	Moyenne (PyTorch)	Très Facile (API)	Difficile

TABLE 1 – Comparaison des modèles pour la détection de fibres

Ensuite, comme nous l'avons expliqué plus tôt dans ce rapport, la configuration des paramètres est un véritable enjeu pour la qualité des résultats obtenus. Cette étape est complètement dépendante de l'échantillon (Sample) étudié, et déterminer une configuration satisfaisante peut être très laborieux. C'est pourquoi nous avons pensé qu'il pourrait être possible d'utiliser des algorithmes de Machine Learning pour optimiser la configuration des paramètres d'entrée.

Cette partie du projet peut être très intéressante et même un sujet à part entière. Nous n'avons donc pas eu le temps de pouvoir implémenter complètement ces pistes pour obtenir des résultats présentables.

V. Conclusion

En l'état, le projet est significativement avancé. Le principal objectif, qui était de parvenir à automatiser la détection des zones d'orientation préférentielle, a été globalement atteint. Nous parvenons désormais à produire des livrables et des rendus satisfaisants. Cependant, la qualité des résultats dépend de plusieurs critères critiques :

- **Visibilité des fibres** : il est impératif que les images étudiées montrent les fibres apparentes. Le type d'images requises correspond typiquement à celles issues du dossier « *PSYN-2022* ».
- **Netteté des images** : il faut limiter les zones de flou pour optimiser l'efficacité du programme.

De plus, le sujet étant tellement vaste, il reste plusieurs axes de développement sur lesquels nous voulions avancer :

- **Analyse des résultats** : le but étant de quantifier la qualité des résultats obtenus avec notre méthode par rapport à ceux obtenus manuellement, c'est à dire comparer la surface et la position des *Regions* identifiées
- **Statistiques sur les régions** : il serait intéressant d'extraire des données statistiques sur les différentes *Regions*, notamment concernant le nombre ou la longueur des fibres qu'elles contiennent.
- **Utilisation du Machine Learning** : il est nécessaire de continuer à développer l'utilisation du machine learning pour améliorer la détection des fibres mais aussi pour l'optimisation des paramètres.
- **Classe Fret** : un des objectifs était d'analyser les images post-fretage, pour cela il serait possible d'implémenter une classe Fret destiné à décrire ces essais. Elle serait définie principalement par deux *Samples*, le pion et la piste.
- **Revoir l'approche par Split** : Bien que pertinente au début du projet pour appréhender la problématique du projet, cette méthode présente des limites. On risque de « couper » certaines fibres, générant ainsi deux contours pour une même fibre, ce qui pourrait fausser les analyses statistiques des *Regions*.

Références Bibliographiques

- OPENCV (2026a). *Image Thresholding*. URL : https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html (visité le 11/03/2026).
- (2026b). *Smoothing Images*. URL : https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html (visité le 02/03/2026).
- SHAPELY (2026). *Shapely*. URL : <https://shapely.readthedocs.io/en/stable/> (visité le 02/03/2026).
- WIKIPEDIA (2026a). *Alpha Shape*. URL : https://en.wikipedia.org/wiki/Alpha_shape (visité le 02/03/2026).
- (2026b). *Analyse d'images*. URL : https://fr.wikipedia.org/wiki/Analyse_d'image (visité le 11/03/2026).
- (2026c). *DBSCAN*. URL : <https://fr.wikipedia.org/wiki/DBSCAN> (visité le 02/03/2026).
- (2026d). *Détection de contours*. URL : https://fr.wikipedia.org/wiki/D%C3%A9tection_de_contours (visité le 11/03/2026).
- (2026e). *Détection de contours*. URL : https://fr.wikipedia.org/wiki/M%C3%A9thode_d'Otsu (visité le 11/03/2026).
- (2026f). *OpenCV*. URL : <https://fr.wikipedia.org/wiki/OpenCV> (visité le 02/03/2026).
- (2026g). *Programmation Orientée Objet*. URL : https://fr.wikipedia.org/wiki/Programmation_orient%C3%A9e_objet (visité le 02/03/2026).
- (2026h). *Traitement d'images*. URL : https://fr.wikipedia.org/wiki/Traitement_d'images (visité le 11/03/2026).
- (2026i). *Vision par ordianteur*. URL : https://fr.wikipedia.org/wiki/Vision_par_ordinateur (visité le 11/03/2026).

Table des figures

1	Structure du classe	4
2	Résultat du floutage bilatéral sur le <i>split</i> 14/64	6
3	Résultat du seuillage sur le <i>split</i> 14/64.....	8
4	Illustration de l'algorithme <i>DBSCAN</i>	9
5	Application de l'algorithme <i>DBSCAN</i> sur le <i>split</i> 1/128	10
6	Rendu des régions par <i>split</i>	11
7	Mapping par translation	12
8	Diagramme du nombre de fibre par angles	12
9	Diagramme du nombre de fibre par direction principales.....	13
10	Enveloppe convexe d'un nuage de points	13
11	Triangulation de Delaunay	14
12	Illustration de l'algorithme <i>Alpha shape</i>	15
13	Illustration de la fermeture morphologique.....	16
14	Rendu des délimitations par fermeture morphologique	20
15	Rendu des délimitations avec fibres par fermeture morphologique	21
16	Rendu d'une <i>Region</i> seule par fermeture morphologique	22
17	Rendu des délimitations et des fibres avec Alpha Shape.....	23
18	Rendu d'une <i>Region</i> seule avec Alpha Shape	24



B Configuration pour le *sample 25*

```
{
  "description": "input params configuration",
  "compute_params": {
    "sample_index"      : "25",
    "nb_split"          : 64,
    "blur_method"       : "BILATERAL",
    "thresh_method"     : "CLASSIC",
    "fiber_perimeter_min" : 10,
    "fiber_len_min"      : 20,
    "fiber_width_max"    : 20,
    "fiber_ratio_min"    : 2,
    "region_min_fiber"   : 100,
    "region_min_area"    : 500000,
    "hole_min_area"     : 100000,
    "delta_angle"       : 8.85,
    "shape_method"      : "MORPH",
    "r_dilate"          : 400,
    "r_erode"           : 30,
    "alpha"             : 0.0065,
    "point_step"        : 200
  },
}
```



```
"render_params":{  
    "render"          : "FIBER_SHAPE",  
    "fill_shape"      : false,  
    "shape_thickness" : 30,  
    "fiber_thickness" : 3,  
    "background"      : "black"  
}
```

isae

supméca

Institut supérieur de mécanique de Paris
3 rue Fernand Hainaut 93407 Saint-Ouen Cedex
tél. 01 49 45 29 00 / www.isae-supmeca.fr

Ministère de l'Enseignement supérieur,
de la Recherche et de l'Innovation

GROUPE
isae 